WEEK 03

Views, Identity, and Safe Change

Views, generated identifiers, and a safe change to the support database.

03

Metro Support: three kinds of records

Table	One row represents	Starting rows
users	One resident or staff member	8
tickets	One support request	12
ticket_events	One event in a request history	21

Active status: new, open, or in_progress. Seven starting tickets are active.

A requester and an assignee are different roles

Ticket	Requester	Assigned staff
1001: streetlight	101: Requester 101	201: Agent 201
1004: broken bench	104: Requester 104	NULL: not assigned

requester_id and assignee_id both refer to users.user_id.
Only the assignment is optional.

Choose a workspace; load the data once

Route	Where PostgreSQL runs	What you open
Colab notebook	In a temporary notebook runtime	Week 3 notebook: setup first
Supabase SQL Editor	In your personal hosted project	SQL Editor, not Table Editor
PGlite playground	In your browser	The SQL prompt

```
SELECT current_database(), current_user;
```

Load once. Keep the Day 1 view for Day 2. A reset deletes that practice state.

Check the starting data before creating a view

```
SELECT 'users' AS table_name, count(*) FROM metro_support.users
UNION ALL
SELECT 'tickets', count(*) FROM metro_support.tickets
UNION ALL
SELECT 'ticket_events', count(*) FROM metro_support.ticket_events;

SELECT ticket_id, requester_id, assignee_id, status
FROM metro_support.tickets WHERE ticket_id = 1004;
```

Counts: 8 / 12 / 21. Ticket 1004: requester 104, assignee NULL, status new.

A view gives a query a reusable name

Object	What PostgreSQL retains	What a reader receives
Base table	The stored ticket rows	The columns and rows requested
Ordinary view	A SELECT definition	A result from the current visible data
Materialized view	Previously computed result rows	Stored results until a refresh

Our active queue is an ordinary view. Resolving a ticket changes which rows its next query can return.

A worked view: active tickets and requesters

```
CREATE OR REPLACE VIEW
  metro_support.active_ticket_summary AS
SELECT t.ticket_id, t.subject, t.priority,
       u.display_name AS requester_name
FROM metro_support.tickets AS t
JOIN metro_support.users AS u
  ON u.user_id = t.requester_id
WHERE t.status IN
  ('new', 'open', 'in_progress');
```

Each ticket has one required requester. The join retrieves that person's current display name.

The lab instead adds an optional assignee. That relationship needs a LEFT JOIN.

CREATE saves the definition. A separate SELECT displays the result.

Reading through the named view

```
SELECT ticket_id, requester_name
FROM metro_support.active_ticket_summary
ORDER BY ticket_id;
```

Seven active tickets. The same requester can appear on more than one ticket.

ORDER BY belongs in the reading query when the display needs a predictable order.

Ticket	Requester
1001	Requester 101
1002	Requester 102
1004	Requester 104
1006	Requester 102
1007	Requester 103
1009	Requester 101
1011	Requester 103

An existing view has a column contract

Current lab view	Append one column	Preserve
ticket_id, subject, status, priority, opened_at	assignee_name goes last	Existing names, order, and types

PostgreSQL expands SELECT * when it creates a view. A later table column does not automatically appear in that existing view.

CREATE OR REPLACE VIEW can append columns. Inserting one in the middle can fail or require a new interface.

The connection also has an identity

```
SELECT current_database(), current_user,
       current_schema();
SHOW search_path;
```

current_user is the effective PostgreSQL role. It is different from the generated row identifier we study next.

metro_support.tickets names the intended schema and table. Your hosted project and role names may differ from the demonstration.

An identity column generates a row identifier

```
CREATE TABLE metro_support.demo_change_notes (
  note_id bigint GENERATED BY DEFAULT
    AS IDENTITY PRIMARY KEY,
  change_name text NOT NULL,
  created_at timestamptz NOT NULL DEFAULT now()
);
```

Omitting note_id asks its sequence for a number. PRIMARY KEY separately requires a unique, non-null key.

Create this demonstration table once. The lab uses its own table, change_notes.

Three inserts, with an explicit decision each time

```
BEGIN;
INSERT INTO metro_support.demo_change_notes (change_name)
VALUES ('Initial view') RETURNING note_id;
COMMIT;

BEGIN;
INSERT INTO metro_support.demo_change_notes (change_name)
VALUES ('Cancelled change') RETURNING note_id;
ROLLBACK;

BEGIN;
INSERT INTO metro_support.demo_change_notes (change_name)
VALUES ('Approved revision') RETURNING note_id;
COMMIT;
```

Run the complete batch. A fresh demonstration table allocates 1, 2, and 3.

A rolled-back row can leave an identifier gap

INSERT returns	Transaction outcome	Retained row
1	COMMIT	1: Initial view
2	ROLLBACK	No row for the cancelled change
3	COMMIT	3: Approved revision

```
SELECT note_id, change_name
FROM metro_support.demo_change_notes ORDER BY note_id;
```

Final row count: 2. Largest ID: 3. A sequence does not promise consecutive committed identifiers.

Lab 1: A queue and a generated identifier

Individual work in your own course database

1. Create the supplied queue and append assignee_name.
   Preserve the two unassigned tickets with a LEFT JOIN.

2. Run the identity experiment and explain the gap in the IDs.

Brightspace: week_03_views_identity.sql
Keep explanations as comments in this one file.

DAY 2

A new field without invented history

Day 2: add source_channel while preserving old rows and older applications.

One change affects old data and old software

State	Historical tickets	New writers	Older writers
Before change	No structured channel	Cannot record the channel	Write the old column list
After change	unknown unless verified	Supply web, phone, or mobile	Omitted channel defaults to unknown

A note mentioning a mobile form does not justify labeling every historical ticket web.

This is a 12-row practice migration. Large live tables also require a lock and deployment plan.

The precheck establishes our starting state

```
SELECT column_name
FROM information_schema.columns
WHERE table_schema = 'metro_support'
  AND table_name = 'tickets'
  AND column_name = 'source_channel';

SELECT count(*) FROM metro_support.tickets;
SELECT count(*) FROM metro_support.active_ticket_queue;
```

Expected:

0 column rows
12 tickets
7 active tickets

Continue from Lab 1. Do not reset the dataset between the two labs. If the column exists, inspect the earlier attempt.

A complete rehearsal for the required field

```
BEGIN;
ALTER TABLE metro_support.tickets
  ADD COLUMN source_channel text;
UPDATE metro_support.tickets SET source_channel = 'unknown'
  WHERE source_channel IS NULL;
ALTER TABLE metro_support.tickets
  ADD CONSTRAINT tickets_source_channel_allowed
  CHECK (source_channel IN ('web', 'phone', 'mobile', 'unknown'));
ALTER TABLE metro_support.tickets
  ALTER COLUMN source_channel SET NOT NULL,
  ALTER COLUMN source_channel SET DEFAULT 'unknown';
SELECT source_channel, count(*)
FROM metro_support.tickets GROUP BY source_channel;
ROLLBACK;
```

Inside the transaction: unknown, 12. ROLLBACK removes the column and its new rules.

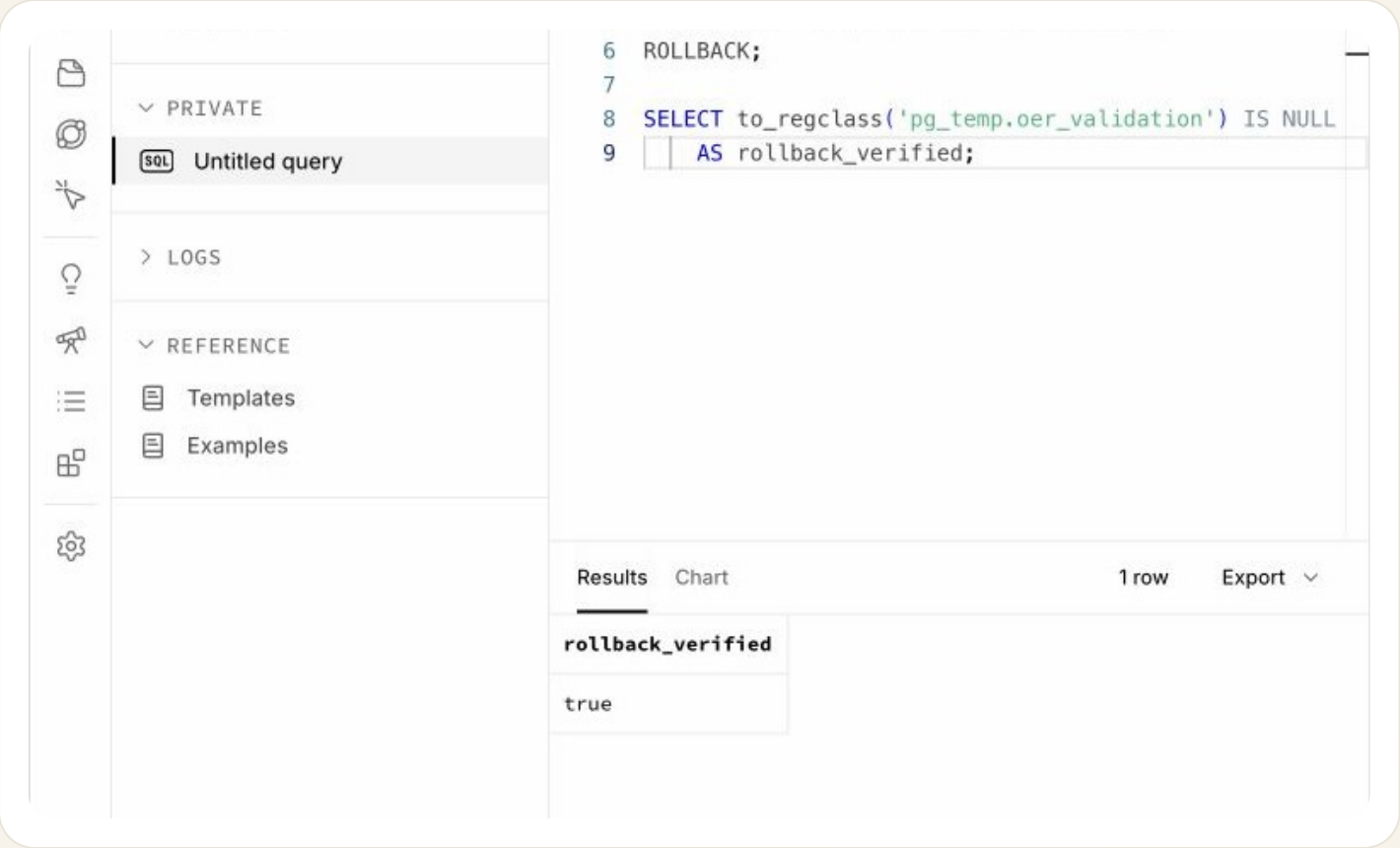
Verification checks the intended change

Check	Inside rehearsal	After ROLLBACK
Column and rules exist	source_channel has its CHECK, default, NOT NULL	No source_channel column
Historical row meaning	12 rows contain unknown	The original rows remain
Old queue still works	7 active tickets	7 active tickets

```
SELECT column_name FROM information_schema.columns
WHERE table_schema = 'metro_support' AND table_name = 'tickets'
  AND column_name = 'source_channel';
```

SUPABASE SQL EDITOR

A temporary-table rollback in Supabase



1. **The saved screenshot shows a disposable temporary-table check.**
2. rollback_verified = true means that temporary table is absent.
3. The ticket migration needs its own column and row checks.

After commit, the view can expose the new field

Repeat the verified migration with COMMIT in place of ROLLBACK, then run:

```
CREATE OR REPLACE VIEW metro_support.active_ticket_queue AS
SELECT t.ticket_id, t.subject, t.status, t.priority, t.opened_at,
       u.display_name AS assignee_name, t.source_channel
FROM metro_support.tickets AS t
LEFT JOIN metro_support.users AS u ON u.user_id = t.assignee_id
WHERE t.status IN ('new', 'open', 'in_progress');
```

The first six columns keep their names and positions. source_channel becomes column seven.

The LEFT JOIN still preserves unassigned tickets 1004 and 1009.

One accepted value and one rejected value

```
BEGIN;
UPDATE metro_support.tickets
SET source_channel = 'mobile' WHERE ticket_id = 1004
RETURNING ticket_id, source_channel;
ROLLBACK;
```

RETURNING shows mobile. Rollback restores unknown.

```
UPDATE metro_support.tickets
SET source_channel = 'fax' WHERE ticket_id = 1004;
```

Run the invalid UPDATE separately. Expected: tickets_source_channel_allowed, SQLSTATE 23514.

Rollback and later repair have different limits

When	What we can do	What must survive
Before COMMIT	ROLLBACK the rehearsal	The original rows and interface
After new mobile data arrives	Repair the view or writer while keeping channel data	Newly collected channel values

DEFAULT unknown supports an older writer that omits the field. An explicit NULL still fails NOT NULL.

A transaction can hold locks while it is open. Keep the practice batch together and finish it.

Lab 2: Add a field without inventing history

Individual work, continuing in the same database

1. Confirm the starting state and complete the three SQL rules.

2. Rehearse, roll back, apply, and append the field to the view.

3. Test valid and invalid values. Explain unknown and the repair limit.

Brightspace: week_03_safe_migration.sql
One SQL file with short comments. No separate change report.

A change is ready when its callers still work

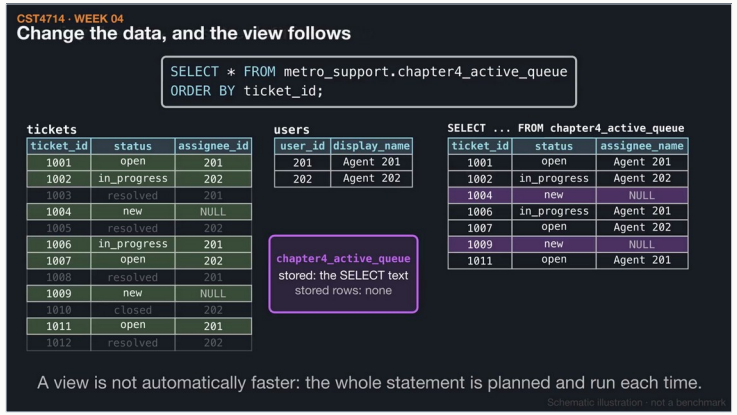
We can explain the stored definition, the changed data, and what the application will receive.

The queue still includes unassigned work.

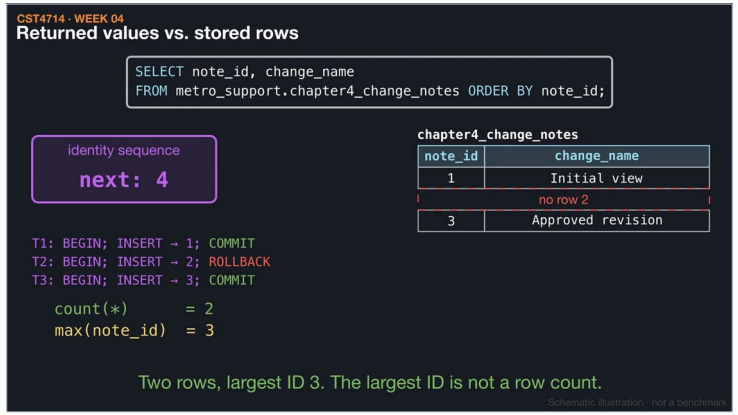
Generated identifiers distinguish records without counting them.

The new channel rule accepts known values without inventing old facts.

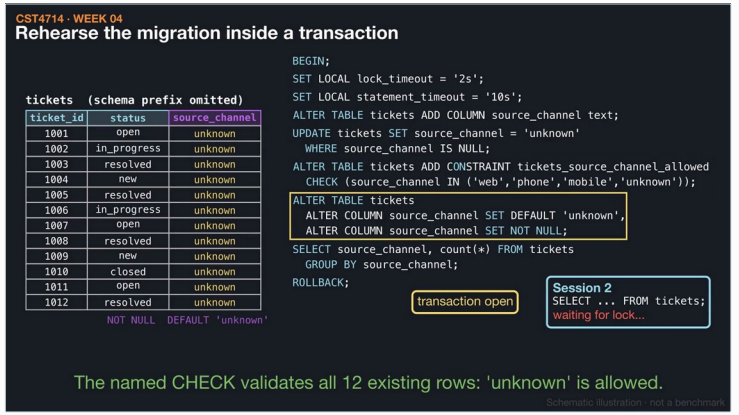
This week's videos



▶ A view stores a query, not rows (1:14)



▶ Why identity values have gaps (1:24)



▶ Adding a required field without inventing history (1:27)

In PowerPoint Slide Show, click a thumbnail to play full screen. Embedded videos work offline.